Ders 13 Genişletilmiş Notu: Hash Fonksiyonları, Doğum Günü Paradoksu, Anahtar Türleri ve Açık Adresleme

Ders 13 Genişletilmiş Notu: Hash Fonksiyonları, Doğum Günü Paradoksu, Anahtar Türleri ve Açık Adresleme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Hashing Tekrarı
- 2. Çarpma Yöntemi
- 3. Doğum Günü Paradoksu ve Çakışma Olasılığı
- 4. Anahtar Türleri
- 5. Çakışma Problemine Çözümler
- 6. Doğrusal Sınama
- 7. Tablonun Dolması, Yeniden Hashleme ve Silme
- 8. Öbeklenme ve İkinci Dereceden Sınama
- 9. Çift Hashing
- 10. Örnek Sorular

1. Hashing Tekrarı

Dersin ilk bölümü sınav ve ödev duyurularına ayrıldı. Konu anlatımı yaklaşık 00:19:40'ta başladı. Geçen hafta elektrik kesintisi yüzünden ders yarım kaldığı için bu haftanın uygulama saati de konu anlatımına ayrıldı. Hoca, hash konusunun bu hafta bütünüyle bitirileceğini söyledi.

Hoca önce geçen haftayı hızlıca özetledi. Arama işlemlerinin hangi veri yapılarıyla yapılabileceği ve karmaşıklıkları konuşulmuştu: ikili arama (binary search), ikili arama ağacı (binary search tree) gibi. Hashing yöntemiyle aramanın $O(1)$ karmaşıklıkta yapılabildiği, bu yüzden anahtar-değer (key-value) biçiminde saklanabilen pek çok bilginin hash yöntemiyle saklandığı söylenmişti.

Doğrudan adres tablosu (direct address table), yani diziyi doğrudan anahtarla indekslemek, normal şartlarda mümkündür. Ama eleman uzayı büyüdükçe dizinin pek çok gözü kullanılmaz. Çok büyük adres değerleri gelince küçük adresler boş kalır. Bu yüzden doğrudan adres tablosu yalnız değer aralığı küçük olduğu bilinen, belli boyuttaki veri setleri için kullanılabilir.

1.1 Hashing'in Adımları

Hashing'de bir anahtar (key) değeri kullanılır ve bu değer hash fonksiyonuna verilir. Anahtarın genelde benzersiz (unique) bir kimlik olması gerekir. Ama her zaman öyle olmayabilir. Örneğin sözlük (dictionary) mantığında anahtar olarak sözcükler kullanılır ve bu sözcüklerden bir değer üretilir.

Hocanın örneği öğrenci bilgisiydi. Öğrenci numarası hash fonksiyonuna verilir. Fonksiyon bir adres üretir. Hash tablosunun o gözüne gidilir ve anahtarın karşılığı olan değer (örneğin öğrencinin notu) oraya yazılır.

Hash tablosu üzerinde üç temel işlem yapılır: arama (search), ekleme (insert) ve silme (delete).

1.2 İyi Bir Hash Fonksiyonunun Özellikleri

- **Kolay hesaplanabilirlik (easy to compute):** Fonksiyon mümkün olduğunca az işlemle hesaplanmalıdır.
- **Tutarlılık (consistency):** Burada kastedilen, tablonun verimli kullanılmasıdır; kararlılık ve doluluk anlamında. Tablonun çok fazla boş kalması, tablo boyutunun yanlış seçildiği anlamına gelir.
- **Düzgün adres üretimi (uniform address generation):** Değerler tabloya mümkün olduğunca dağınık, yayılmış biçimde yerleşmelidir.

Geçen hafta $m = 10$ seçilmişti. Bu durumda 10, 40, 50, 100 gibi değerlerin hepsi 0 adresini üretir ve tablonun belli bir bölgesi çok şişer. Bunu istemeyiz. Bu yüzden tablo boyutunun asal sayı (prime number) seçilmesi genelde tercih edilir.

Saklanacak eleman sayısı n ile tablo boyutu m arasında bir ilişki vardır. n/m oranı yük faktörünü (load factor) verir:

$$\alpha = \frac{n}{m}$$

Hoca, bu yük faktörüne göre bazı kararlar verileceğini ve bunun ilerleyen bölümde görüleceğini söyledi.

1.3 Bölme Yöntemi Hatırlatması

Bölme yöntemi (division method) modüler aritmetik kullanır: $h(k) = k \bmod m$. Geçen haftaki örnekte $m = 11$ için $25 \bmod 11 = 3$ bulunmuş ve 25, tablonun 3 numaralı gözüne yazılmıştı. Hoca, orada doğrudan 25'in saklandığını, ama başka bir değer de saklanabileceğini hatırlattı.

Bölme ve çarpma yöntemlerinin ortak dezavantajı şudur: Eleman sayısı arttıkça çakışma (collision) olasılığı da artar. Hoca bunu hashing'in en büyük dertlerinden biri olarak tanımladı. Hashing çok verimli bir arama karmaşıklığına sahiptir, ama çakışma problemiyle karşı karşıyadır. Dersin geri kalanı bu problemle ilgilidir.

2. Çarpma Yöntemi

Çarpma yönteminde (multiplication method) $0 < A < 1$ aralığında bir A sabiti kullanılır:

$$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$$

Hoca "$kA \bmod 1$"in ne anlama geldiğine yakından baktı. Bu ifade kA'nın kesir kısmıdır:

$$kA \bmod 1 = kA - \lfloor kA \rfloor$$

Yani önce anahtar A ile çarpılır. Sonucun kesir kısmı alınır; bu değer 0 ile 1 arasındadır. Bu kesir tablo boyutu m ile çarpılır ve tam kısmı adres olarak alınır.

A için 0 ile 1 arasında herhangi bir değer seçilebilir. Ama en çok kullanılan değer şudur:

$$A = \frac{\sqrt{5} - 1}{2} \approx 0.6180339887$$

Hoca bu sabitin düzgün dağılım için kullanılan en uygun değerlerden biri olduğunu söyledi.

2.1 Sayısal Örnek

Verilen: $m = 10000$, $k = 123456$, $A \approx 0.6180339887$.

- $kA = 123456 \cdot 0.6180339887 \approx 76300.0041151$
- Kesir kısmı: $kA \bmod 1 \approx 0.0041151$
- $m \cdot 0.0041151 = 41.151$
- Tam kısmı alınır: $h(123456) = 41$

> **Ders sonrası not:** Hoca tahtaya A'yı "0.618033" olarak yazdı, ama verdiği 76300.0041151 sonucu A'nın tam değeriyle ($0.6180339887\ldots$) elde edilir. Python ile kontrol: tam A ile $h(123456) = 41$ çıkar. Altı basamaklı 0.618033 ile çarpım 76299.882 olur ve adres değişir. Bu yöntemde A'nın basamakları sonucu doğrudan etkiler.

2.2 Adres Tablo Boyutunu Geçebilir mi?

Hoca çarpma yönteminin bölme yöntemine göre artısını ya da eksisini sordu. İlk gelen cevap bir soruydu: Anahtar çok büyük olursa üretilen adres tablo boyutunu geçebilir mi?

Soru: Anahtar değeri çok yüksek olursa adres tablonun boyutunu geçebilir mi? **Cevap:** Geçemez. kA mod 1 her zaman 1'in altında bir değerdir. m (örnekte 10.000) 1'den küçük bir değerle çarpılınca sonuç hiçbir zaman m'yi geçmez. Adres her zaman $0 \leq h(k) \leq m - 1$ aralığındadır.

2.3 Bölme Yöntemiyle Karşılaştırma

Hoca, iyi bir hash fonksiyonunun "kolay hesaplanabilir" olması gerektiğini hatırlatarak ipucu verdi: Hangisi daha az işlem gerektirir?

Çarpma yönteminde daha fazla işlem yapılır: çarpma, kesir kısmını alma, yeniden çarpma ve tam kısmı alma. Bu yüzden çarpma yöntemi hashing'de çok sık kullanılmaz. Öte yandan dağılımın düzgünlüğü açısından bakılırsa, çarpma yöntemi biraz daha düzgün değerler üretebilir.

Hoca karşılaştırmayla ilgili bir uyarı da yaptı. Bölme yönteminde seçilen asal sayı önemlidir; tablo boyutu büyüdükçe dağılım da doğal olarak iyileşir. Çarpma yöntemindeki m de aynı şekilde önemlidir. Bu yüzden iki yöntemi karşılaştırırken m'lerin aynı seviyede olması gerekir. Çok küçük bir m ile çok büyük bir m karşılaştırılırsa aynı şeyler karşılaştırılmış olmaz.

3. Doğum Günü Paradoksu ve Çakışma Olasılığı

3.1 Problem

Hoca doğum günü paradoksunu (birthday paradox) bir soruyla açtı: Yılın 365 gün olduğunu düşünürsek, bir ortamda bulunan iki kişinin doğum günlerinin aynı olma olasılığı nedir?

Bir öğrenci $1/365$ cevabını verdi. Hoca cevabı revize etmeyi önerdi ve soruyu tersinden sordu: Farklı olma olasılığı nedir?

- **2 kişi:** Farklı olma olasılığı $1 - \frac{1}{365} = \frac{364}{365}$.
- **3 kişi:** $\frac{364}{365} \cdot \frac{363}{365}$. Payda 365^2 olur.
- k **kişi:** Bir öğrenci genel terimi şöyle kurdu: 364'ten başlayıp her terimde bir azalarak son terimde $365 - (k - 1)$ yazılır.

$$P(\text{k kişinin doğum günleri farklı}) = \frac{364}{365} \cdot \frac{363}{365} \cdots \frac{365 - (k-1)}{365}$$

Hoca son terimi parantez içinde $365 - (k - 1)$ olarak yazdı, "daha iyi formüle etmek için".

> **Ders sonrası not:** İki kişi için öğrencinin verdiği "aynı olma olasılığı $1/365$" cevabı doğrudur. Hocanın revize önerisi, cevabı yanlış saymaktan çok, k kişilik genellemeye "farklı olma" olasılığı üzerinden geçmek içindi. Python ile: 2 kişi için aynı olma olasılığı $1 - 364/365 \approx 0.00274 = 1/365$.

3.2 Hash Tablosuna Uyarlama

Bu formül hashing'de şu soruya karşılık gelir: k anahtarın hiç çakışmama olasılığı nedir? Hoca, bunu hesaplamak için hangi bilginin gerektiğini sordu. Bir öğrenci sayıların aralığının (en küçük ve en büyük değerin) gerektiğini söyledi; hoca "daha basit bir şey" dedi. Doğru cevap tablo boyutudur (table size). Doğum günü probleminde tablo boyutunun yerinde 365 vardı.

365 yerine m yazılınca k anahtarın hiç çakışmama olasılığı:

$$\frac{m-1}{m} \cdot \frac{m-2}{m} \cdots \frac{m-(k-1)}{m} = \prod_{i=1}^{k-1} \frac{m-i}{m} \approx e^{-k(k-1)/(2m)}$$

Hash çakışma olasılığı (hash collision probability) bunun 1'den çıkarılmış hâlidir. Bunu bir öğrenci cevapladı:

$$p \approx 1 - e^{-k(k-1)/(2m)}$$

> **Ders sonrası not:** Hoca yaklaşık ifadeyi "e üzeri eksi $k(k-1)$ bölü m" diye söyledi ve çakışma formülünü de bu biçimde yazdı. Doğrusu paydada $2m$ olan $e^{-k(k-1)/(2m)}$'dir. Aşağıdaki "77.163 eklemede %50" sonucu 2'li formülden gelir. 2'siz formül $k = 77163$ için yaklaşık 0.75 verir (Python ile kontrol edildi).
>
> **Ek bilgi (derste anlatılmadı):** Yaklaşım, küçük x için $1-x \approx e^{-x}$ olmasından gelir. Her çarpan $\frac{m-i}{m} = 1 - \frac{i}{m} \approx e^{-i/m}$ yazılır. Üsler toplanınca $\sum_{i=1}^{k-1} \frac{i}{m} = \frac{k(k-1)}{2m}$ elde edilir. Paydadaki 2 bu toplamdan gelir.

3.3 32 Bitlik Hash Değerleri İçin Sayılar

Hoca formülü $m = 2^{32}$ için, yani 32 bitlik hash değerleri üretildiği durum için uyguladı:

Ekleme sayısı k	Çakışma olasılığı p
77.163	yaklaşık %50
250.000	yaklaşık %100

Python ile kontrol: $k = 77163$ için $p \approx 0.49999$; $k = 250000$ için $p \approx 0.99931$. Tam çarpım formülü de aynı sonuçları verir.

> **Ders sonrası not:** Hoca "m'yi 32 üzeri 32 aldığımız bir noktada" dedi. Hemen ardından "32 bitlik hash değerleri" dediği için kastedilen $m = 2^{32}$'dir.

Hoca ilerleyen bölümlerde başarısız arama (unsuccessful search) ve başarılı arama (successful search) gibi işlemlerin maliyetlerinin de hesaplanacağını söyledi.

Soru: 250.000 eklemeden sonra çakışma olasılığı neden %100 oluyor? **Cevap:** Hoca soruyu sınıfa yöneltti. Bir öğrenci "tablo tamamen dolmuş oluyor" dedi. Hoca "yaklaşık olarak tablonun dolduğu bir noktaya ulaşıyoruz" diyerek bu cevabı kabul etti. Formülde k yerine 250.000, m yerine 2^{32} konunca e'li terim 0'a, sonuç 1'e yaklaşır. Benzer şekilde $k = 77163$ konunca e'li terim yaklaşık 0.5 olur ve $p \approx 1 - 0.5 = 0.5$ çıkar.

> **Ders sonrası not:** "Tablo doldu" açıklaması yanlıştır. $m = 2^{32} \approx 4.3$ milyar gözdür. 250.000 anahtar tablonun yalnız yaklaşık %0.006'sını doldurur. Olasılığın 1'e yaklaşmasının nedeni doğum günü paradoksunun kendisidir: Üsteki $k(k-1)$ terimi k ile değil k^2 ile büyür. Kabaca $k \approx \sqrt{m}$ mertebesinde ($\sqrt{2^{32}} = 65536$) çakışma beklenir hâle gelir. %50 eşiği $k \approx \sqrt{2m \ln 2} \approx 77163$'tür.

4. Anahtar Türleri

Hoca, hash fonksiyonuna verilecek anahtarın farklı türlerde olabileceğini anlattı.

4.1 Tam Sayı ve Kesirli Sayı

1. **Tam sayı (integer):** Doğrudan kullanılır.
2. **Kesirli sayı:** Anahtar kesirli bir sayı olarak verilmişse önce tam sayıya dönüştürülmelidir. Hoca bu dönüşümden bizim sorumlu olduğumuzu vurguladı.

4.2 String Anahtarlar

Üçüncü tür string'dir. Hoca tahtaya aynı harflerden oluşan birkaç sözcük yazdı ve sordu: Bu string'ler birbirinden farklıdır ve aynı hash değerini üretmemeleri istenir. Hangi durumda aynı değeri üretirler?

Bir öğrenci cevap verdi: Karakterler toplanırsa, sözcükler aynı karakterlere sahip olduğu için toplamlar ve sonraki işlemler de aynı olur. Hoca bunu doğruladı. Karakterlerin ASCII karşılıkları toplanıp sonra bölme ya da çarpma yöntemi uygulansaydı, bu sözcüklerin hepsi için aynı sayı üretilirdi.

> **Ders sonrası not:** Basit toplamanın aynı sonucu vermesi için sözcüklerin aynı harflerden oluşması (birbirinin harf sırası değiştirilmiş hâli olması) gerekir. Kısa notta örnek olarak verilen "elma", "mali", "alma" bu koşulu sağlamaz: Python ile ASCII toplamları sırasıyla 415, 419 ve 411 çıkar. Tahtadaki sözcüklerin harfleri büyük olasılıkla birebir aynıydı.

Hoca bunun için karakterleri doğrudan toplamayan çözümler anlattı.

Yöntem A. Hoca tahtaya bir formül yazdı ve terimlerin anlamını sordu. Öğrencilerin cevaplarıyla şu yapı ortaya çıktı:

- $s - 1$ benzeri bir terim string'in uzunluğuna (size) karşılık gelir.
- c_0 string'in ilk karakteri, c_1 ikinci karakteridir.
- $\text{ord}(c_i)$, ilgili karakterin ASCII tablosundaki karşılığıdır.
- C, string'de yer alan bütün karakter kodlarından büyük seçilen bir sabittir.
- h sıfırdan başlatılır.

Her turda önceki değer sabitle çarpılıp üzerine yeni karakterin kodu eklenir:

$$h \leftarrow h \cdot C + \text{ord}(c_i), \quad i = 0, 1, \ldots, s - 1$$

Böylece her karakterin katkısı konumuna göre farklı bir ağırlık alır. Aynı harflerden oluşan sözcükler için farklı değer üretilme olasılığı çok yüksek olur.

Yöntem B: Horner yöntemi (Horner's method). Hoca ikinci bir yöntem yazdı. Her karakter için alfabedeki sırası kullanılır: $\text{str}[i] - \text{'a'} + 1$. Böylece alfabenin ilk harfi 1 değerini alır.

$$h \leftarrow h \cdot R + (\text{str}[i] - \text{'a'} + 1), \quad i = 0, 1, \ldots, \text{len} - 1$$

R genelde asal seçilir. Sık kullanılan değerlerden biri 31'dir. Döngü string'in uzunluğu kadar döner.

> **Ek bilgi (derste anlatılmadı):** Horner yöntemi aslında bir polinomun değerini az işlemle hesaplama yöntemidir. Döngü sonunda elde edilen değer $v_0 R^{n-1} + v_1 R^{n-2} + \cdots + v_{n-1}$ polinomudur; burada v_i i. karakterin değeridir. Her adımda yalnız bir çarpma ve bir toplama yapıldığı için kuvvetleri ayrı ayrı hesaplamaya gerek kalmaz.

İki yöntem de string'i doğrudan hash fonksiyonu yapmaz. String'den sayısal bir anahtar üretilir. Sonra bu anahtar bölme ya da çarpma yöntemine verilir ve adres bulunur. Hoca bu ön işlemlerin nedenini şöyle özetledi: Standart toplamayla farklı kelimelerin aynı adresi gösterme ihtimali vardır. A ve B yöntemleri bu ihtimali ortadan kaldırmak için kullanılır.

4.3 Bileşik Anahtar

Dördüncü tür bileşik anahtardır (compound key). Örneğin bir doğum günü (31.08.1997) birden fazla değerden oluşur. Buna göre saklama yapmak için değerler bir sabitin kuvvetleriyle birleştirilir:

$$31 \cdot R + 8 \cdot R^2 + 1997 \cdot R^3$$

R için yine 31 ya da başka bir asal sayı verilebilir. Hoca, bileşik anahtarın tek bir bütünleşik değer olmak zorunda olmadığını da söyledi. Elimizdeki ayrı değerler de birleştirilebilir. Örneğin yaş, kilo ve boy birlikte hashlenmek isteniyorsa, bu yöntem bu anahtarlardan tek bir anahtar üretmenin yollarından biridir.

Aradan sonra hoca şunu ekledi: Pratikte çözümler genelde string ve tam sayı anahtarlar içindir. Bileşik anahtarlarla da karşılaşılır, ama ağırlıklı olarak string ve tam sayılar için bir hash fonksiyonu kullanılır.

4.4 Sabit Uzunluklu String'ler

Soru: Örneklerde hep dört karakterlik string'ler kullanıldı. Her karakter bir bayt olduğuna göre, karakterleri sırayla okuyup kaydırarak (shift) dört baytlık bir tam sayıya yerleştirsek ve bu tam sayıyı anahtar olarak kullansak bu da bir yöntem olabilir mi? **Cevap:** Uygulanabilir bir yöntemdir. Ama hashlemek istediğimiz bilgiler genelde böyle denk gelmez; hepsi dört karakterli olmayabilir. Dört karakter yalnız bir örneği temsil etmek için seçildi. Farklı uzunlukta string'ler olduğunda, örneğin 20 harften oluşan bir kelimede, baytları yan yana ekleme yaklaşımı sorun çıkarır. Girdinin sınırlı olduğu, örneğin en fazla 4 harflik string'lerin kullanıldığı bir ortamda ise baytlar sırayla yazıldığı için farklı değerler üretir ve kullanılabilir.

5. Çakışma Problemine Çözümler

Aradan sonra hoca hashing'in büyük derdi olan çakışma problemine döndü. Çakışmaya karşı iki temel yaklaşım vardır:

1. **Ayrık zincirleme (separate chaining).** Açık hashing (open hashing) olarak da adlandırılır.
2. **Açık adresleme (open addressing).** Kapalı hashing (closed hashing) olarak da adlandırılır.

Hoca bir uyarı yaptı: "Open hashing" ile "open addressing" adları birbirine karışabilir. Açık ve kapalı hashing adlarını bilmek iyidir, ama kullanırken daha rahat etmek için "separate chaining" ve "open addressing" adları tercih edilmelidir.

İki yaklaşımın mantığı da basittir:

- **Açık adresleme:** Hash tablosunda yerleşilmek istenen göz doluysa, bir sonraki uygun göze bakılır.
- **Ayrık zincirleme:** Yeni gelen anahtar dolu bir gözle aynı adresi gösteriyorsa, o gözde bir bağlı liste (linked list) kurulur. Aynı adrese sahip elemanlar bu listede arka arkaya saklanır.

Bu derste açık adresleme işlendi. Ayrık zincirleme sonraki oturumun konusudur.

6. Doğrusal Sınama

6.1 Bir Sonraki Uygun Göz

Hoca sordu: Bir anahtar hash fonksiyonuna verildi, bir adres üretildi ve bu adres dolu. Bir sonraki uygun göze (next available) nasıl bakılabilir? Bir öğrenci cevap verdi: Boş bir indeks bulunana kadar indeks birer artırılır, eleman bulunan boş yere yerleştirilir.

Bu, açık adreslemenin ilk ve en basit çözümü olan doğrusal sınamanın (linear probing) temel mantığıdır:

$$h(k, i) = (h'(k) + i) \bmod m, \quad i = 0, 1, 2, ...$$

Burada $h'(k)$ anahtardan üretilen ilk adrestir; örneklerde $h'(k) = k \bmod m$.

6.2 Ekleme Örneği

Tablonun gözleri $0, 1, \ldots, 6$ olarak numaralandı, yani $m = 7$.

> **Ders sonrası not:** Hoca önce "m'yi 6 seçtim" dedi, sonra hesapları mod7 ile yaptı. Bir öğrencinin sorusu üzerine $m = 7$ seçildiği için tablo boyutunun da 7 olduğunu söyledi. Doğrusu $m = 7$'dir.

Anahtarlar sırayla eklenir:

- **9:** $h(9, 0) = 9 \bmod 7 = 2$. Göz 2 boş, 9 oraya yerleşir.
- **16:** $h(16, 0) = 16 \bmod 7 = 2$. Göz 2 dolu, i bir artırılır. $h(16, 1) = (2 + 1) \bmod 7 = 3$. Göz 3 boş, 16 oraya yerleşir.
- **11, 12, 13:** Sırasıyla $11 \bmod 7 = 4$, $12 \bmod 7 = 5$, $13 \bmod 7 = 6$. Hepsi ilk denemede boş göze yerleşir.
- **23:** Bir öğrenci izledi: $23 \bmod 7 = 2$ dolu. Sonra 3, 4, 5, 6 gözlerine bakılır; hepsi dolu. Mod alındığı için başa dönülür: $(2 + 5) \bmod 7 = 0$. 23, 0 numaralı göze yerleşir.
- **8:** $8 \bmod 7 = 1$. Göz 1 boş, 8 oraya yerleşir.

Hoca, modüler aritmetik sayesinde dairesel (circular) bir yapı kurulduğunu vurguladı.

Göz	0	1	2	3	4	5	6
Anahtar	23	8	9	16	11	12	13

Bu noktada tablo doludur. Ardından 28 ya da 30 gelirse, sınama dizisi bütün gözleri gezer ve boş göz bulunamaz. Bu izleme Python ile doğrulandı.

6.3 Arama

Bir öğrenci aramada sorun çıkıp çıkmayacağını sordu: 16 aranırken ilk bakılan göz 9'un olduğu göz değil mi? Başka bir öğrenci aramanın nasıl yapılacağını açıkladı ve hoca bu açıklamayı doğru buldu:

- Aramaya $i = 0$ ile, anahtarın gösterdiği ilk gözden başlanır.
- Göz doluysa ve içindeki aranan anahtar değilse i bir artırılır.
- Aranan anahtara denk gelinirse arama başarılıdır.
- Boş göze (NULL) denk gelinirse arama başarısızdır; anahtar tabloda yoktur.

Örnekte 16 arandığında önce göz 2'ye bakılır; orada 9 vardır. Sonra göz 3'e bakılır ve 16 bulunur.

6.4 Sonsuz Döngü ve $i < m$ Koşulu

Hoca bu arama mantığını bir akış diyagramı olarak çizdi. Döngüye girmeden önce sayaç sıfırlanır ve ilk adres key mod m olarak hesaplanır. Sonra sordu: Tablo doluyken bu arama işimizi çözer mi?

Bir öğrenci, tablo dolduğu için tablonun büyütülmesi gerektiğini söyledi. Hoca o anda aramadan söz ettiklerini hatırlattı. Öğrenci aramanın döngüye gireceğini söyledi: Tablodaki elemanlar aranınca bulunur, ama tabloda olmayan bir anahtar aranırsa döngü bitmez. Başka bir öğrenci çözümü verdi: Kuyruk ve yığında yaptığımız gibi, i'nin bütün tabloyu dolaşıp kendi üzerimize geri gelip gelmediğini m ile kontrol etmeliyiz.

Hoca bunu doğruladı. Döngü koşuluna "$i < m$ ve" eklenir. Bu koşul konunca arama en fazla m adımda tamamlanır. Koşul konmazsa, tablo doluyken ve anahtar yokken arama sonsuz döngüye girer, çünkü ne anahtar ne de boş göz bulunur.

> **Hocanın vurgusu:** Bu kontrolü koymazsanız arama işlemi sonsuz bir döngüye girecektir.

Aşağıdaki C kodu, akış diyagramının $i < m$ koşulu eklenmiş karşılığıdır (diyagramdan kuruldu; derlenip yukarıdaki tabloyla denendi: 16 için 3, 23 için 0, 28 için -1 döner):

```c
#include <stdio.h>

#define M 7
#define EMPTY 0

int table[M];

/* Aranan anahtarın gözünü döndürür, yoksa -1 döndürür. */
int search(int key) {
    int i = 0;
    int adr = key % M;
    while (i < M && table[adr] != EMPTY && table[adr] != key) {
        i++;
        adr = (key % M + i) % M;
    }
    if (i < M && table[adr] == key) {
        return adr;
    }
    return -1;
}
```

Ekleme işleminde de durum aynıdır. Her turda bakılan göz doluysa eleman yerleştirilemez. Tablo doluysa yine sonsuz döngü başlar. Bunu kırmanın yolu, i tablo boyutuna ulaştığında eklemenin yapılamadığını kullanıcıya hata olarak bildirmektir.

7. Tablonun Dolması, Yeniden Hashleme ve Silme

7.1 Tablo %80-90 Dolunca

Hoca, tablonun dolma durumunun zaten istenmediğini söyledi. Bunun önceden tespit edilmesi gerekir. Tablo %80 ya da %90 dolduysa bir çözüm uygulanmalıdır. Hoca bu çözümün ne olduğunu sordu.

Soru: Tablo %80-90 dolunca ne yapılmalıdır? **Cevap:** Bir öğrenci ayrık zincirlemeyi önerdi. Hoca bunun tablonun dolduğu durumda kullanılabilecek bir yöntem ve kabul edilebilir bir cevap olduğunu söyledi. Ama açık adreslemeyle devam edebilmek için asıl yapılması gerekeni sordu. Bir öğrenci "tabloyu büyütmek" fikrini fazla bencilce bulduğunu yazdı. Başka bir öğrenci tablonun büyütülemeyeceğini düşündü. Hoca, öyle bir kısıt koymadığını söyledi.

Hocanın vardığı sonuç şudur: Bu kaçınılmaz bir sondur. Tablo boyutundan daha fazla eleman saklamak gerekiyorsa ilk çözüm bağlı liste, yani ayrık zincirleme kullanmak gibi görünür. Ama açık adresleme kullanıldıysa ve bir doyum noktasına ulaşıldıysa yeniden hashleme (rehashing) yapılır:

- Daha büyük bir tablo açılır.
- Küçük tablodaki değerler yeniden bir hash fonksiyonundan geçirilerek büyük tabloya aktarılır.
- Elemanlar yeni pozisyonlarına yerleşir. Aynı pozisyona yerleşme garantisi yoktur; hepsi farklı bir yere yerleşebilir.

  Hocanın vurgusu: Açık adreslemede tablo dolma noktasına geliyorsa genelde yeniden hashleme yapıyoruz.

7.2 Silme

Soru: Açık adreslemede eleman silindiğinde ne yapılır? **Cevap:** Bir öğrenci, oraya elemanın silindiğine dair bir işaret konacağını söyledi. Hoca "aynen öyle" dedi. İşaret olarak örneğin -1 kullanılabilir. Bu bir silme bayrağı (delete flag) olarak göze yerleştirilir.

Hoca ardından sordu: Belli bir süre sonra silme işlemleri arttı ve epey eleman silindi. O zaman ne yapılmalı? Bir öğrenci boşlukları dolduracak şekilde kaydırma (shift) önerdi. Hoca, bu noktada tablonun yeniden tasarlanması (redesign) gerektiğini söyledi. Tablonun verimliliğini

artırmaya yönelik bir yaklaşım uygulanmalıdır. Bu, aynı tablo üzerinde değil, yeni bir tablo açmak anlamına da gelebilir.

> **Ek bilgi (derste anlatılmadı):** Silinen gözün boş (NULL) bırakılmaması gerekir, çünkü arama boş gözde "anahtar yok" diye durur. Silinen gözün ilerisine yerleşmiş bir anahtar, örneğin tablodaki 16, 9 silinip göz 2 boşaltılsaydı bulunamazdı. Silme işaretli göz aramada "dolu" gibi atlanır, eklemede ise "boş" gibi kullanılabilir.

8. Öbeklenme ve İkinci Dereceden Sınama

8.1 Öbeklenme

Hoca doğrusal sınamanın bir problemini anlattı: öbeklenme (clustering). Doğrusal sınamada $+1, +1$ adımlarla gidilir. Belirli bölgelerde üretilen adresler birbirine yakınsa, yani adresler düzgün üretilmediyse, anahtarlar arka arkaya birikmeye başlar. Tabloya istenen şekilde yayılmazlar. Rastgele dağılması beklenen adresler, doğrusal artım yüzünden ardışık gözlere yerleşir. Belirli bölgelerde çakışma olursa bu bölgeler öbeklenir.

8.2 İkinci Dereceden Sınama

Hoca çözümü sordu ve ipucu verdi: Eklenen i üzerinden düşünebilirsiniz. Bir öğrenci i yerine i^2 eklenmesini önerdi. Bu, ikinci dereceden sınamadır (quadratic probing):

$$h(k, i) = (h'(k) + i^2) \bmod m$$

i yerine i^2 eklenince elemanlar arka arkaya değil, $+1, +4, +9$ şeklinde ilerleyerek yerleşir. Böylece tabloya daha geniş bir şekilde yayılır.

Örnek ($m = 7$): Tabloda 2 ve 3 numaralı gözler önceden dolu. $k = 9$ eklenecek.

- $i = 0$: $h(9, 0) = 9 \bmod 7 = 2$. Dolu.
- $i = 1$: $h(9, 1) = (2 + 1^2) \bmod 7 = 3$. Dolu.
- $i = 2$: $h(9, 2) = (2 + 2^2) \bmod 7 = 6$. Boş, 9 göz 6'ya yerleşir.

> **Ders sonrası not:** Tahtada üçüncü satır "hash(9,1)" olarak yazılmıştı. Bir öğrenci düzeltti, hoca da kabul etti: Doğrusu "hash(9,2)"dir.

8.3 Genel Biçim

İkinci dereceden sınamada iki sabitli genel bir biçim de kullanılabilir. Bu biçim doğrusal sınamanın terimini de devreye sokar:

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

Örnek: $c_1 = 2$, $c_2 = 1$, $k = 9$, $i = 1$ için

$$h(9, 1) = (2 + 2 \cdot 1 + 1 \cdot 1^2) \bmod 7 = 5$$

Hoca, iki farklı katsayıyla bir hash fonksiyonu kurmanın tabloyu belli oranda daha etkili kullanmayı sağladığını söyledi. Bu sonuçlar Python ile doğrulandı.

8.4 Kareler Üzerinde Öbeklenme

Soru: İkinci dereceden sınamayla ardışık indislenmeyi önlemeye çalışıyoruz. Ama aynı adrese düşen çok sayıda anahtar gelirse ve hepsi için i^2 kullanılırsa, bu kez ardışık gözler yerine kareler üzerinde öbeklenme olmaz mı? **Cevap:** Bakış açısı doğrudur. Ama elimizdeki değerlerin ne olduğunu bilmiyoruz; aynı adrese düşenler varsayımıyla böyle bir denk gelme olasılığından söz ettik. Doğrusal sınamada sıra 2, 3, 4, 5, 6 diye giderken, ikinci dereceden sınamada başka bir anahtar 3'e değil daha uzak gözlere yerleşir. Örüntü dizinin içine yayılır ve yeni gelen bir

anahtarın aynı gözü gösterme olasılığı azalır. Aynı değerler gelecekse, onlar için yine i'nin karelerini arka arkaya hesaplayıp peşinden gitmek zorundayız; bunun için yapılabilecek bir şey yoktur. Avantaj, toplam veri setinin yerleştirilmesindedir: O grup yine bir örüntüyle yerleşir, ama geri kalanlar daha dengeli dağılır.

> **Ek bilgi (derste anlatılmadı):** Literatürde doğrusal sınamadaki ardışık birikmeye birincil öbeklenme (primary clustering), aynı ilk adrese düşen anahtarların aynı sınama dizisini izlemesine ikincil öbeklenme (secondary clustering) denir. İkinci dereceden sınama birincil öbeklenmeyi azaltır, ikincil öbeklenmeyi gidermez. Çift hashing ikisini de azaltır, çünkü adım büyüklüğü anahtara bağlıdır.

9. Çift Hashing

Açık adresleme altındaki son yaklaşım çift hashingdir (double hashing). İki hash fonksiyonu kullanılır:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

- $h_1(k) = k \bmod m$; m tablo boyutudur.
- $h_2(k) = 1 + (k \bmod m')$ ve $m' < m$.

9.1 Örnek

Verilen: $m = 13$, $h_1(k) = k \bmod 13$, $h_2(k) = 1 + (k \bmod 11)$. 14 sayısı eklenecek.

Bir öğrenci önce şöyle hesapladı: $14 \bmod 13 = 1$ ve $14 \bmod 11 + 1 = 3 + 1 = 4$. Toplam 5. Sonra tabloda bazı sayıların önceden yerleşmiş olduğu varsayıldı ve 5'in dolu olduğu görüldü. Başka bir öğrenci devam etti: i bir artırılır, $2 \cdot 4 = 8$, $8 + 1 = 9$. Göz 9 boş olduğu için 14, 9 numaralı göze yerleşir.

Bir öğrenci sonradan hesabındaki bir atlamayı düzeltti, hoca da teşekkür etti. Doğru izleme $i = 0$'dan başlar:

- $h_1(14) = 14 \bmod 13 = 1$, $h_2(14) = 1 + (14 \bmod 11) = 1 + 3 = 4$
- $i = 0$: $(1 + 0 \cdot 4) \bmod 13 = 1$. Dolu.
- $i = 1$: $(1 + 1 \cdot 4) \bmod 13 = 5$. Dolu.
- $i = 2$: $(1 + 2 \cdot 4) \bmod 13 = 9$. Boş, 14 göz 9'a yerleşir.

Sonuç aynı kalır, ama i normalde 0'dan başlatılır. Hoca bu kısmı uzatmamak için bilerek ayırmadığını söyledi. Bu izleme Python ile doğrulandı.

9.2 Örüntüyü Bozmak

Hoca farkı şöyle özetledi: İkinci dereceden sınamada i kendisiyle çarpılıyordu; i^2 ya da i^3 de kullanılabilir, ama onların da bir örüntüsü vardır. Önemli olan bu örüntüyü bozmaktır. Çift hashingde i, ikinci hash fonksiyonunun sonucuyla çarpılır. Bu biraz daha maliyetlidir, ama örüntüyü bozmak açısından daha etkili bir çözümdür ve daha düzgün bir dağılım sağlar.

9.3 m' Neden m'den Küçük?

Soru: İkinci hash fonksiyonundaki modül m' neye göre seçilir? Neden m'den küçük yazıldı?
Cevap: Hoca, m''nin m'den küçük olması gerektiğini, böylece bütün tablonun verimli kullanılacağını söyledi. m' m'den büyük olursa tablonun dışında bir değer üretme olasılığı çıkar. m'den küçük rastgele bir değer seçilebilir, ama asal sayı seçmek daha akıllıcadır. Örneğin $m = 31$ ise $m' = 7$; örnekteki gibi 13 ve 11.

> **Ders sonrası not:** "Tablonun dışında değer üretir" açıklaması tam doğru değildir. Sonuçta mod$m$ alındığı için adres her zaman tablonun içindedir. Asıl nedenler şunlardır:
>
> - $h_2(k)$ hiçbir zaman 0 ya da m'nin katı olmamalıdır; yoksa adım büyüklüğü sıfır olur ve hep aynı göze bakılır. $h_2(k) = 1 + (k \bmod m')$ ve $m' < m$ seçilince $1 \le h_2(k) \le m - 1$ olur.

- $h_2(k)$ ile m aralarında asal olmalıdır; böylece sınama dizisi bütün gözleri gezer. m asal seçilirse bu koşul kendiliğinden sağlanır.

10. Örnek Sorular

Bu derste ekran karesi kullanılmadı. Aşağıdaki sorular derste sözlü olarak sorulanlardır.

1. (derste sözlü, 00:29:38) Çarpma yönteminin bölme yöntemine göre artısı ya da eksisi var mıdır?

> **Çözüm:** Çarpma yönteminde daha çok işlem vardır; "kolay hesaplanabilirlik" açısından bölme yöntemi öndedir ve çarpma yöntemi pratikte daha az kullanılır. Buna karşılık çarpma yöntemi, özellikle $A = (\sqrt{5}-1)/2$ ile, biraz daha düzgün dağılım verebilir. Karşılaştırmada iki yöntemin m'si aynı seviyede olmalıdır.

2. (derste sözlü, 00:30:14) Çarpma yönteminde anahtar çok büyükse üretilen adres tablo boyutunu geçebilir mi?

> **Çözüm:** Geçemez. $kA \bmod 1 < 1$ olduğu için $m \cdot (kA \bmod 1) < m$ olur.

3. (derste sözlü, 00:34:32) Yılın 365 gün olduğu düşünülürse bir ortamdaki iki kişinin doğum günlerinin aynı olma olasılığı nedir? Farklı olma olasılığı nedir? Üç kişi ve k kişi için farklı olma olasılığı nedir?

> **Çözüm:** Aynı olma: $1/365$. Farklı olma: $364/365$. Üç kişi: $\frac{364}{365} \cdot \frac{363}{365}$. k kişi: $\frac{364}{365} \cdot \frac{363}{365} \cdots \frac{365-(k-1)}{365}$.

4. (derste sözlü, 00:39:14) k sayının çakışmama olasılığını hesaplamak için hangi parametreye ihtiyaç vardır? Bu olasılık nedir?

> **Çözüm:** Tablo boyutu m gerekir. Olasılık $\prod_{i=1}^{k-1} \frac{m-i}{m} \approx e^{-k(k-1)/(2m)}$'dir.

5. (derste sözlü, 00:40:28) Hash çakışma olasılığı nedir?

> **Çözüm:** Çakışmama olasılığının 1'den çıkarılmış hâlidir: $p \approx 1 - e^{-k(k-1)/(2m)}$.

6. (derste sözlü, 00:42:58) $m = 2^{32}$ için 250.000 eklemeden sonra çakışma olasılığı neden yaklaşık %100 olur?

> **Çözüm:** Derste "tablo doluyor" cevabı kabul edildi; bu cevap yanlıştır (bkz. Bölüm 3.3'teki ders sonrası not). Formülde $k(k-1)/(2m) = 250000 \cdot 249999/2^{33} \approx 7.28$ olur, $e^{-7.28} \approx 0.0007$ ve $p \approx 0.9993$ çıkar. Tablonun yalnız yaklaşık %0.006'sı doludur; olasılık k^2 ile büyüdüğü için 1'e yaklaşır.

7. (derste sözlü, 00:45:47) Aynı harflerden oluşan farklı string'ler hangi durumda aynı hash değerini üretir? Bunu önlemek için ne yapılır?

> **Çözüm:** Karakterlerin ASCII kodları toplanıp bölme ya da çarpma yöntemi uygulanırsa aynı değeri üretirler, çünkü toplama karakterlerin sırasını dikkate almaz. Önlemek için her karakter konumuna göre ağırlıklandırılır: $h \leftarrow h \cdot C + \text{ord}(c_i)$ ya da Horner yöntemiyle $h \leftarrow h \cdot R + (\text{str}[i] - \text{'a'} + 1)$, genelde $R = 31$.

8. (derste sözlü, 01:19:36) $m = 7$ olan doğrusal sınamalı tabloda 2-6 numaralı gözler doluyken 23 nereye yerleşir? Ardından 8 de eklenip tablo dolduktan sonra 28 ya da 30 gelirse ne olur?

> **Çözüm:** $23 \bmod 7 = 2$; 2, 3, 4, 5, 6 dolu, mod ile başa dönülür ve 23 göz 0'a yerleşir. 8 göz 1'e yerleşir ve tablo dolar. 28 ve 30 için sınama dizisi bütün gözleri gezer, yer bulunamaz. Ekleme $i = m$ olunca "tablo dolu" hatası vermelidir. Asıl çözüm tablo %80-90 dolmadan yeniden hashleme yapmaktır.

9. (derste sözlü, 01:22:32) Doğrusal sınamalı aramanın akış diyagramı tablo doluyken işimizi çözer mi? Sonsuz döngü nasıl engellenir?

Çözüm: Çözmez. Tabloda olmayan bir anahtar aranırsa ne anahtar ne boş göz bulunur ve döngü bitmez. Döngü koşuluna $i < m$ eklenir; arama en fazla m adımda biter.

10. (derste sözlü, 01:29:13) Açık adreslemede eleman silindiğinde ne yapılır? Silmeler çok artarsa ne yapılmalıdır?

Çözüm: Göze bir silme işareti (örneğin -1) konur. Silmeler çok artarsa tablo yeniden düzenlenir; gerekirse yeni bir tablo açılır.

11. (derste sözlü, 01:32:25) Doğrusal sınamadaki öbeklenme problemi nasıl azaltılır?

Çözüm: i yerine i^2 eklenir (ikinci dereceden sınama): $h(k, i) = (h'(k) + i^2) \bmod m$. Adımlar $+1, +4, +9$ olur ve anahtarlar tabloya daha geniş yayılır.

12. (derste sözlü, 01:39:28) $m = 13$, $h_1(k) = k \bmod 13$, $h_2(k) = 1 + (k \bmod 11)$ olan çift hashingde 14 nereye yerleşir? (1 ve 5 numaralı gözler dolu.)

Çözüm: $h_1(14) = 1$, $h_2(14) = 4$. $i = 0$: göz 1 dolu; $i = 1$: göz 5 dolu; $i = 2$: $(1 + 8) \bmod 13 = 9$. 14 göz 9'a yerleşir.

Dersin Toparlaması

Bu derste hash fonksiyonlarının ikinci yöntemi olan çarpma yöntemi işlendi. Bölme yöntemine göre daha çok işlem gerektirdiği, ama biraz daha düzgün dağılım verebildiği konuşuldu. Doğum günü paradoksu üzerinden çakışma olasılığı $p \approx 1 - e^{-k(k-1)/(2m)}$ olarak kuruldu. 32 bitlik bir tabloda bile yaklaşık 77 bin eklemede çakışma olasılığının %50'ye ulaştığı görüldü. Bu, çakışmanın kaçınılmaz olduğunu ve bir çözüm yöntemi gerektiğini gösterir. Anahtar türleri olarak tam sayı, kesirli sayı, string ve bileşik anahtar ele alındı. String'lerde karakterleri basitçe toplamak yerine konuma bağlı yöntemler, özellikle Horner yöntemi anlatıldı.

Dersin ikinci yarısında çakışma çözümleri ayrık zincirleme ve açık adresleme olarak ikiye ayrıldı ve açık adresleme işlendi. Doğrusal sınamada aramanın boş gözde durduğu, tablo doluyken sonsuz döngüyü önlemek için $i < m$ koşulunun şart olduğu, tablo %80-90 dolunca yeniden hashleme yapıldığı ve silinen gözlere işaret konduğu konuşuldu. Doğrusal sınamanın öbeklenme sorununa karşı ikinci dereceden sınama, örüntüyü daha iyi bozan bir yöntem olarak da çift hashing anlatıldı. Hoca, uygulama oturumunda ayrık zincirleme ile açık hashing farkının, evrensel hashing (universal hashing), yük faktörü ve başarılı/başarısız arama maliyetlerinin konuşulacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.